

Royaume du Maroc
Ministère de l'Éducation Nationale, du Préscolaire et des Sports
Académie Régionale de l'Éducation et de la Formation Drâa-Taflalet
Direction Provinciale de Ouarzazate



Rapport de Projet de Fin d'Études

Deuxième année BTS
Développement des Applications Informatiques

**Conception et réalisation d'une application mobile de
gestion des commandes d'un restaurant**

Mon PF App

Projet	Mon PF App
Réalisé par	HAJJI Yassmine
Établissement	Lycée Qualifiant Technique Ibn Al-Haitam
Formation	BTS Développement des Applications Informatiques
Encadrement	Encadrant pédagogique : à compléter
Année de formation	2024 – 2026

Remerciements

Je tiens à remercier l'équipe pédagogique du BTS Développement des Applications Informatiques du Lycée Qualifiant Technique Ibn Al-Haitam pour l'encadrement, les connaissances techniques et la méthodologie acquises durant ma formation.

Je remercie également toutes les personnes qui ont contribué aux tests, aux remarques et à l'amélioration progressive de l'application **Mon PF App**. Ce projet m'a permis de rapprocher les acquis de programmation, base de données, analyse UML, sécurité, tests et documentation d'un besoin applicatif concret : la gestion des commandes d'un restaurant.

Résumé

Ce rapport présente **Mon PF App**, une application mobile et multi-plateforme de gestion des commandes d'un restaurant. Le système permet à un client de consulter un menu, gérer un panier, passer une commande et suivre son état. Il propose aussi des espaces dédiés à l'administrateur, au serveur et au livreur.

La solution repose sur **Flutter** pour l'interface Android, Windows et Web, et sur **Laravel** pour l'API REST. La version livrée est une version **démo-prod** adaptée à la maintenance : backend local, base SQLite seedée, comptes de test, sécurité des rôles, tests automatiques, guide HTML interactif et rapport LaTeX.

Table des matières

Remerciements	i
Résumé	ii
1 Présentation	1
1.1 Fiche signalétique	1
1.2 Intérêt du sujet	1
1.3 Organisation du rapport	1
2 Contexte et spécification générale	2
2.1 Contexte du projet	2
2.2 Problématique	2
2.3 Objectifs du projet	2
2.4 Spécification fonctionnelle générale	2
2.5 Besoins non fonctionnels	3
3 Modélisation UML	4
3.1 Présentation d'UML	4
3.2 Diagrammes UML employés	4
3.3 Cas d'utilisation	4
3.4 Diagramme de classes du système	4
3.5 Diagramme de séquence : passer une commande	5
4 Architecture technique et technologies utilisées	7
4.1 Étude technique	7
4.2 Critères de sélection	7
4.3 Langages de développement	8
4.4 Frameworks et outils	8
4.5 Sécurité technique	8
5 Réalisation	9
5.1 Architecture de l'application	9
5.2 Réalisation backend	9
5.3 Réalisation frontend	9
5.4 Guide interactif et assistant debug	10
5.5 Structure détaillée du frontend Flutter	10
5.6 Structure détaillée du backend Laravel	11
5.7 Description des interfaces graphiques	11
5.8 Gestion des rôles	12

5.9	Algorithmes et règles métier	12
5.9.1	Calcul du total du panier	12
5.9.2	Création d'une commande	13
5.10	Tests et validation	13
5.11	Mise en service locale	13
5.12	Scénario de démonstration	14
5.13	Guide de dépannage intégré	14
6	Conclusion	15
6.1	Bilan	15
6.2	Difficultés rencontrées	15
6.3	Validation du cahier de charge	15
6.4	Perspectives	16
A	Comptes de démonstration	17
B	Fichiers importants	18

1 Présentation

1.1 Fiche signalétique

TABLE 1.1 – Fiche signalétique du projet

Élément	Description
Projet	Mon PF App
Sujet	Application mobile de gestion des commandes d'un restaurant
Réalisé par	HAJJI Yassmine
Formation	BTS Développement des Applications Informatiques
Établissement	Lycée Qualifiant Technique Ibn Al-Haitam
Cibles	Android, Windows et Web pour la démonstration
Période	Année de formation 2024 – 2026

1.2 Intérêt du sujet

Le secteur de la restauration utilise de plus en plus les outils numériques pour recevoir les commandes, réduire les erreurs et améliorer la communication avec les clients. Une application de commande constitue donc un sujet pertinent pour un PFE, car elle combine interface mobile, base de données, API REST, sécurité et gestion des rôles.

Le projet mobilise plusieurs compétences de la formation :

- analyse du besoin et rédaction du cahier de charge ;
- modélisation UML et conception de données ;
- développement mobile avec Flutter ;
- développement backend avec Laravel ;
- tests, build Android et documentation.

1.3 Organisation du rapport

La structure du rapport suit la logique académique observée dans les rapports de PFE fournis comme références : présentation, contexte et spécification générale, modélisation UML, architecture technique, réalisation, conclusion et perspectives.

2 Contexte et spécification générale

2.1 Contexte du projet

Dans un restaurant, la prise de commande peut devenir difficile lorsque plusieurs clients commandent en même temps. Les erreurs de saisie, les oublis de plats, les retards de préparation et le manque de suivi entraînent une mauvaise expérience client.

L'application proposée répond à ce contexte en centralisant le menu, le panier, les commandes et les statuts. Elle permet au client de voir ce qu'il commande, au restaurant de recevoir les commandes et au livreur de connaître les livraisons disponibles.

2.2 Problématique

Comment digitaliser le parcours de commande d'un restaurant afin de réduire les erreurs humaines, améliorer le suivi des commandes et fournir une interface simple aux clients, administrateurs, serveurs et livreurs ?

2.3 Objectifs du projet

L'objectif général est de réaliser une application fonctionnelle et présentable pour une démonstration PFE, avec un backend local et des données persistantes.

Les objectifs spécifiques sont :

- permettre aux clients de consulter le menu ;
- gérer un panier et calculer automatiquement le total ;
- passer une commande réelle via API ;
- suivre le statut d'une commande ;
- proposer un espace administrateur et serveur ;
- proposer un espace livreur ;
- sécuriser l'inscription publique en imposant le rôle client côté serveur ;
- fournir un guide de lancement pour Android, Windows et Web.

2.4 Spécification fonctionnelle générale

TABLE 2.1 – Besoins fonctionnels par acteur

Acteur	Fonctionnalités attendues
Client	Créer un compte, se connecter, consulter le menu, ajouter des plats au panier, passer une commande, suivre son état.
Administrateur	Voir les statistiques, consulter les commandes, gérer le menu, affecter un livreur, contrôler les rôles sensibles.
Serveur	Participer au traitement opérationnel des commandes côté restaurant.
Livreur	Voir les commandes disponibles, accepter une livraison et mettre à jour son avancement.

2.5 Besoins non fonctionnels

- **Sécurité** : authentification, token et contrôle d'accès par rôle ;
- **Simplicité** : interface claire pour faciliter la prise en main ;
- **Rapidité** : accès rapide au menu et au panier ;
- **Fiabilité** : données seedées et tests automatiques ;
- **Maintenabilité** : séparation frontend, backend et documentation.

3 Modélisation UML

3.1 Présentation d'UML

UML est un langage de modélisation utilisé pour représenter les acteurs, les cas d'utilisation, les classes, les relations et les scénarios d'un système. Dans ce projet, UML sert à clarifier le fonctionnement avant et pendant la réalisation.

3.2 Diagrammes UML employés

Les diagrammes retenus sont :

- le diagramme de cas d'utilisation pour identifier les actions de chaque acteur ;
- le diagramme de classes pour représenter les entités principales ;
- le diagramme de séquence pour expliquer le passage d'une commande.

3.3 Cas d'utilisation

TABLE 3.1 – Cas d'utilisation principaux

Acteur	Cas d'utilisation
Client	S'inscrire, se connecter, consulter le menu, gérer le panier, valider une commande, suivre le statut.
Administrateur	Consulter le tableau de bord, voir les commandes, gérer le menu, affecter un livreur.
Serveur	Suivre les commandes côté restaurant, modifier certains statuts.
Livreur	Consulter la file de livraison, accepter une commande, marquer l'avancement.

3.4 Diagramme de classes du système

Le modèle de données repose sur les entités `User`, `Restaurant`, `Category`, `MenuItem`, `Order` et `OrderItem`. Le client est relié aux commandes, une commande contient plusieurs lignes et chaque ligne référence un plat.

Modele logique simplifie

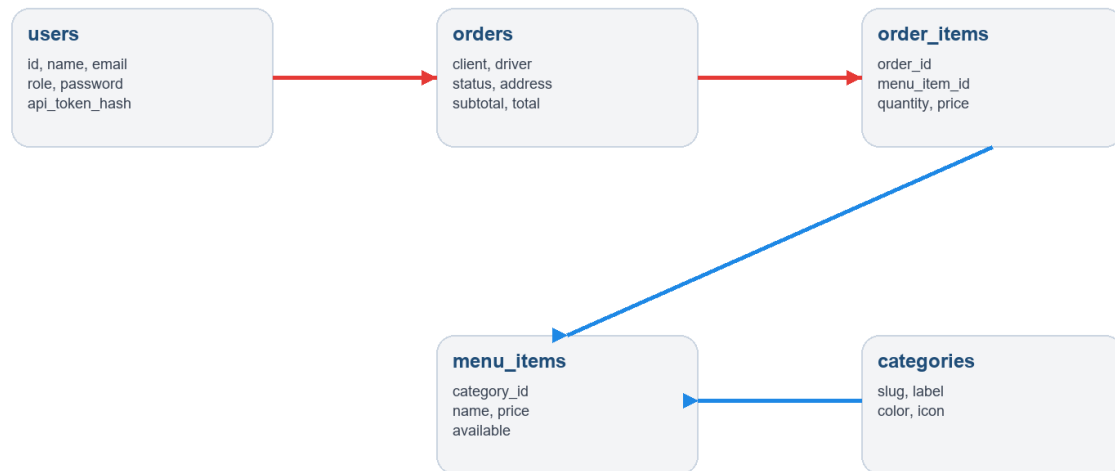
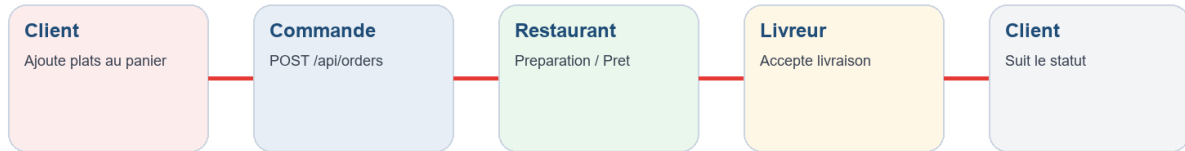


FIGURE 3.1 – Diagramme logique des classes et relations backend

3.5 Diagramme de séquence : passer une commande

Le scénario principal commence par la consultation du menu et se termine par la création d'une commande en base de données. Flutter prépare le panier, Laravel valide les données, puis l'API retourne la commande créée.

Cycle de commande



Regle: une commande ne peut pas être créée sans plats. Les statuts sont contrôlés par l'API.

FIGURE 3.2 – Séquence simplifiée du cycle de commande

4 Architecture technique et technologies utilisées

4.1 Étude technique

L'étude technique a pour objectif de choisir une architecture simple à lancer en soutenance, mais suffisamment structurée pour représenter une application réelle. Le choix s'est porté sur Flutter pour l'interface et Laravel pour le backend.

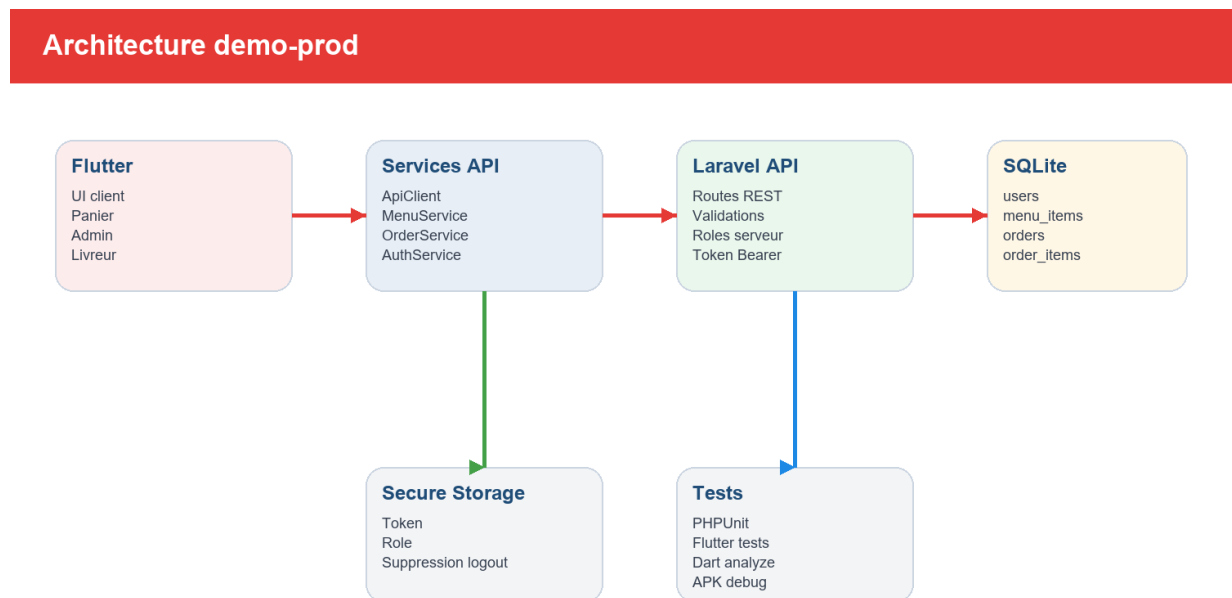


FIGURE 4.1 – Vue d'ensemble de l'architecture démo-prod

4.2 Critères de sélection

Les critères utilisés pour choisir les technologies sont :

- facilité de mise en place sur la machine de démonstration ;
- clarté du code pour un projet BTS ;
- compatibilité Android, Windows et Web ;
- possibilité de tests automatiques ;

— documentation et communauté disponibles.

4.3 Langages de développement

TABLE 4.1 – Langages utilisés

Langage	Utilisation
Dart	Développement de l'application Flutter.
PHP	Développement de l'API Laravel.
SQL	Définition et manipulation de la base SQLite.
HTML/CSS/JavaScript	Guide interactif local et assistant debug pour Yassmine.

4.4 Frameworks et outils

TABLE 4.2 – Choix techniques

Couche	Technologie	Justification
Frontend	Flutter	Interface unique pour Android, Windows et Web.
Backend	Laravel 13	Routes REST, validations, tests, structure MVC.
Base démo	SQLite	Démarrage rapide sans serveur de base externe.
Sécurité mobile	<code>flutter_secure_storage</code>	Stockage plus sûr du token local.
Tests backend	PHPUnit	Vérification automatique des flux API.
Documentation	LaTeX + HTML	Rapport académique et guide interactif.

4.5 Sécurité technique

Les décisions de sécurité importantes sont :

- le rôle n'est plus choisi depuis l'inscription publique ;
- le backend force le rôle `client` pour `/api/register` ;
- les routes sensibles sont protégées par middleware ;
- les tokens sont stockés côté Flutter avec `flutter_secure_storage` ;
- l'URL API est fournie par `API_BASE_URL` au lieu d'être codée en dur ;
- CORS est activé pour la démonstration Web locale.

5 Réalisation

5.1 Architecture de l'application

Le dépôt est organisé en trois zones principales :

- `mon_pfapp` : application Flutter ;
- `mon_pfapi` : API Laravel ;
- `docs` : cahier de charge, rapports, tutoriels et guide HTML.

5.2 Réalisation backend

Le backend expose des endpoints REST pour authentifier les utilisateurs, récupérer le menu, créer des commandes et gérer les statuts.

TABLE 5.1 – Endpoints API principaux

Méthode	Endpoint	Rôle
GET	<code>/api/health</code>	Vérifier que l'API fonctionne.
GET	<code>/api/menu</code>	Récupérer le restaurant, les catégories et les plats.
POST	<code>/api/register</code>	Créer un compte client.
POST	<code>/api/login</code>	Authentifier et retourner un token.
GET	<code>/api/orders</code>	Lister les commandes selon le rôle.
POST	<code>/api/orders</code>	Créer une commande depuis le panier.
PATCH	<code>/api/orders/{order}/status</code>	Mettre à jour le statut.
GET	<code>/api/admin/stats</code>	Consulter les statistiques.
GET	<code>/api/driver/orders</code>	Voir la file de livraison.

5.3 Réalisation frontend

L'application Flutter contient des écrans pour l'authentification, l'accueil, le menu, le panier, le suivi, le profil, l'espace livreur et l'espace admin. Des images de plats et un logo personnalisé remplacent l'interface Flutter par défaut.

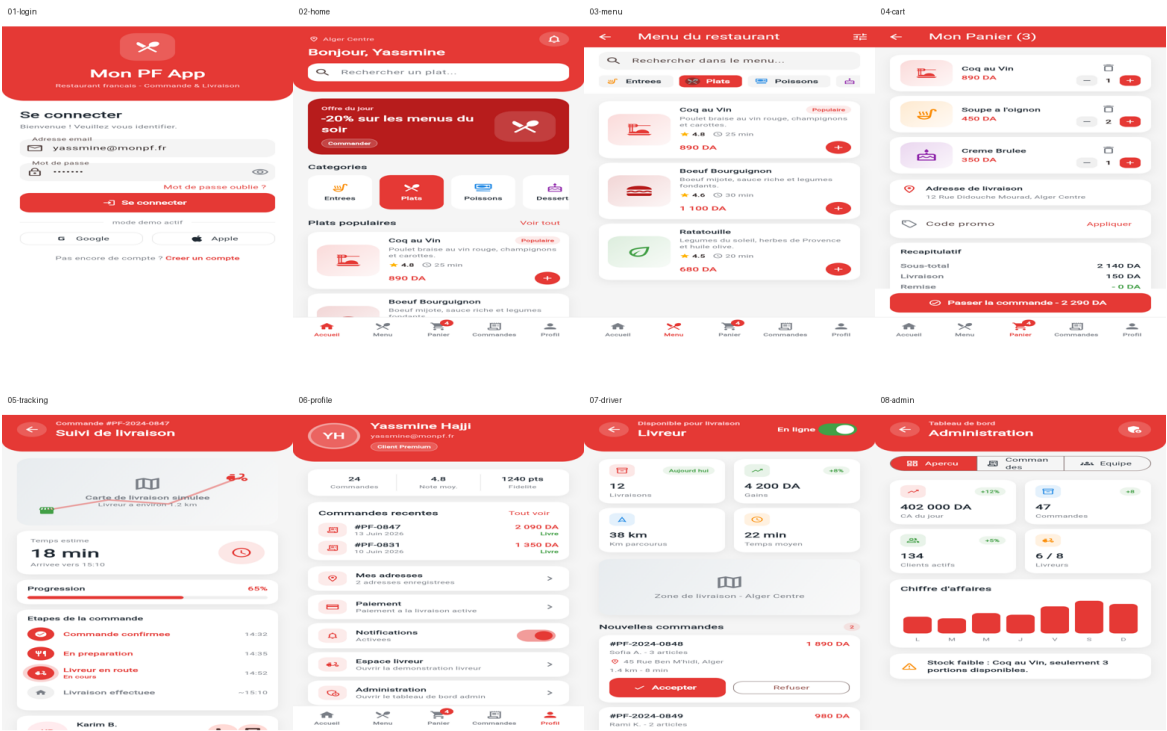


FIGURE 5.1 – Aperçu des interfaces Flutter

5.4 Guide interactif et assistant debug

Un guide HTML local a été ajouté dans docs/yassmine-guide/index.html. Il sert de support pratique pour Yassmine : lancement Android, Windows, Web, utilisation de l’application, erreurs fréquentes et assistant debug basé sur Gemini API.

L’assistant ne contient aucune clé API dans le code source. La clé Gemini est saisie dans le navigateur uniquement au moment de l’utilisation. Si aucune clé n’est fournie, une base locale de solutions répond aux erreurs courantes.

5.5 Structure détaillée du frontend Flutter

La structure Flutter a été organisée pour séparer les responsabilités. Cette séparation facilite la lecture du code, les tests et les futures améliorations.

TABLE 5.2 – Organisation du code Flutter

Dossier ou fichier	Rôle dans l’application
lib/app/mon_pf_app.dart	Point central de l’application : thème, navigation, état utilisateur, panier et chargement des données.
lib/core/config/api_config.dart	Configuration du mode démo et de l’URL API via dart-define.
lib/core/storage/secure_token_store.dart	Stockage sécurisé du token d’authentification.

<code>lib/data/api_client.dart</code>	Client HTTP commun : headers, token, timeout et messages d'erreur.
<code>lib/features/auth/</code>	Écrans et services de connexion, inscription et déconnexion.
<code>lib/features/client/</code>	Écrans client : accueil, menu, panier, suivi et profil.
<code>lib/features/admin/</code>	Écrans administrateur et serveur : statistiques, commandes et menu.
<code>lib/features/driver/</code>	Écrans livreur : file de livraison et acceptation.
<code>lib/shared/widgets/</code>	Composants réutilisables : cartes, boutons, navigation et visuels.

5.6 Structure détaillée du backend Laravel

Le backend Laravel suit une organisation MVC. Les routes reçoivent les requêtes HTTP, les contrôleurs appliquent les règles métier, les modèles manipulent les données et les migrations définissent le schéma.

TABLE 5.3 – Organisation du backend Laravel

Dossier ou fichier	Rôle
<code>routes/api.php</code>	Déclare les endpoints REST publics et protégés.
<code>app/Http/Controllers/Api/AuthController.php</code>	Gère l'inscription, la connexion et la déconnexion.
<code>app/Http/Controllers/Api/MenuController.php</code>	Fournit le catalogue restaurant et les actions de gestion menu.
<code>app/Http/Controllers/Api/OrderController.php</code>	Gère la création, la consultation, les statuts et l'affectation des commandes.
<code>app/Http/Middleware/TokenAuth.php</code>	Vérifie le token Bearer et les rôles autorisés.
<code>database/migrations/</code>	Définit les tables de la base SQLite.
<code>database/seeder/DatabaseSeeder.php</code>	Crée les comptes de démonstration, le restaurant, les catégories et les plats.
<code>tests/Feature/ApiFlowTest.php</code>	Vérifie les flux API critiques.

5.7 Description des interfaces graphiques

TABLE 5.4 – Écrans principaux de l'application

Écran	Description
-------	-------------

Connexion	Permet à un utilisateur existant d'accéder à son espace selon son rôle.
Inscription	Crée un compte client. Le rôle n'est pas choisi dans l'interface publique.
Accueil client	Présente l'identité visuelle du restaurant et l'accès rapide au menu.
Menu	Affiche les catégories et les plats avec images, prix et ajout au panier.
Panier	Récapitule les plats, quantités, sous-totaux et total général.
Suivi	Affiche les commandes du client et leurs statuts.
Dashboard admin	Montre les statistiques utiles et la liste des commandes.
Espace livreur	Présente les commandes disponibles pour livraison et l'action d'acceptation.

5.8 Gestion des rôles

Le système distingue quatre rôles : client, administrateur, serveur et livreur. Cette séparation répond à une règle importante : chaque utilisateur doit uniquement accéder aux actions qui correspondent à son métier.

TABLE 5.5 – Contrôle d'accès par rôle

Action	Rôles autorisés	Protection
Consulter le menu	Public	Route publique.
Créer une commande	Client connecté	Middleware token.
Lister toutes les commandes	Admin, serveur	Middleware rôle.
Modifier le statut	Admin, serveur, livreur selon cas	Middleware + validation métier.
Voir la file livreur	Admin, livreur	Middleware rôle.
Créer un administrateur	Non public	Action non exposée à l'inscription.

5.9 Algorithmes et règles métier

5.9.1 Calcul du total du panier

Listing 5.1 – Principe du calcul de total

```
total = 0
pour chaque ligne du panier:
    total = total + (prix_unitaire * quantite)
```

5.9.2 Création d'une commande

1. Flutter vérifie que le panier n'est pas vide ;
2. le panier est transformé en JSON ;
3. Laravel vérifie l'utilisateur connecté ;
4. Laravel valide les plats et les quantités ;
5. Laravel crée la commande et ses lignes ;
6. Flutter affiche le suivi de commande.

5.10 Tests et validation

TABLE 5.6 – Tests réalisés

Zone	Commande	Objectif
Backend	<code>artisan test</code>	Tester menu, login, inscription client et commande.
Flutter	<code>flutter test</code>	Tester les flux auth/navigation.
Dart	<code>dart analyze</code>	Détecter les erreurs statiques.
Android	<code>flutter build apk</code>	Produire un APK installable.
LaTeX	<code>pdflatex</code>	Générer le rapport académique.

5.11 Mise en service locale

Listing 5.2 – Démarrer Laravel

```
cd C:\Users\LOQ\Documents\yassmin pfe\mon_pfapi
..\tools\runtime\php-8.4.22\php.exe artisan migrate:fresh --seed
..\tools\runtime\php-8.4.22\php.exe artisan serve --host=0.0.0.0
--port=8000
```

Listing 5.3 – Lancer Flutter Windows

```
cd C:\Users\LOQ\Documents\yassmin pfe\mon_pfapp
flutter run -d windows --dart-define=DEMO_MODE=false --dart-define=API_BASE_URL=http://127.0.0.1:8000/api
```

Listing 5.4 – Lancer Flutter Web

```
flutter run -d chrome --web-port=5600 --dart-define=DEMO_MODE=false --dart-define=API_BASE_URL=http://127.0.0.1:8000/api
```

Listing 5.5 – Lancer Flutter Android réel

```
flutter run -d <device-id> --dart-define=DEMO_MODE=false --dart-define=API_BASE_URL=http://ADRESSE_IP_PC:8000/api
```

5.12 Scénario de démonstration

Le scénario recommandé pour la soutenance permet de montrer rapidement toute la chaîne fonctionnelle :

1. démarrer l'API Laravel et tester `/api/health` ;
2. lancer Flutter sur Android, Windows ou Web ;
3. se connecter avec le compte client `yassmine@monpf.fr` ;
4. consulter le menu et ajouter plusieurs plats au panier ;
5. valider la commande et afficher le suivi ;
6. se connecter avec le compte administrateur ;
7. montrer les statistiques et les commandes ;
8. se connecter avec le compte livreur ;
9. accepter une commande disponible ;
10. conclure en expliquant l'architecture Flutter + Laravel.

5.13 Guide de dépannage intégré

Le guide HTML contient un assistant qui aide Yassmine à comprendre les erreurs d'installation. Il ne remplace pas la documentation technique, mais il donne une explication guidée : signification de l'erreur, cause probable dans le projet, solution pas à pas et commande de vérification.

Les erreurs couvertes localement sont notamment :

- Android ne détecte pas le téléphone ;
- l'application Android n'arrive pas à joindre l'API ;
- Windows demande le fichier `atlstr.h` ;
- Web bloque les requêtes à cause de CORS ;
- Laravel n'est pas lancé ou le port 8000 est indisponible.

6 Conclusion

6.1 Bilan

Mon PF App répond au cœur du cahier de charge : authentification, menu, panier, commande, suivi, rôles et livraison. La version obtenue est adaptée à une soutenance PFE car elle présente une interface moderne, un backend fonctionnel, des données locales seedées, des tests et une documentation complète.

Le projet a permis de mettre en pratique les compétences de la formation BTS DAI : analyse, conception, base de données, développement mobile, API REST, sécurité, tests, documentation et mise en service.

6.2 Difficultés rencontrées

TABLE 6.1 – Difficultés et solutions

Difficulté	Cause	Solution
API locale sur téléphone	127.0.0.1 pointe vers le téléphone	Utiliser l'IP Wi-Fi du PC.
Web et CORS	Origines différentes entre Chrome et Laravel	Ajouter <code>HandleCors</code> et <code>config/cors.php</code> .
Sécurité des rôles	Rôle sensible exposé à l'inscription	Imposer le rôle côté serveur.
Stockage token	Stockage simple insuffisant	Utiliser <code>flutter_secure_storage</code> .
Windows ATL	Composant Visual Studio absent	Installer C++ ATL dans Visual Studio Installer.

6.3 Validation du cahier de charge

TABLE 6.2 – Couverture du cahier de charge

Besoin	Statut	Preuve
--------	--------	--------

Authentification	Validé	Login/register API + Flutter.
Menu restaurant	Validé	Endpoint <code>/api/menu</code> .
Panier	Validé	Interface client.
Création commande	Validé	POST <code>/api/orders</code> .
Suivi commande	Validé	Liste et statut.
Admin / serveur	Validé pour démonstration	Dashboard et commandes.
Livreur	Validé pour démonstration	File de livraison et acceptation.
Guide d'installation	Validé	Guide HTML et tutoriel PDF.
Paiement en ligne	Hors périmètre	Perspective.
Notifications push	Hors périmètre	Perspective.

6.4 Perspectives

Pour passer d'une version de démonstration à une production réelle, les améliorations suivantes sont recommandées :

- héberger l'API sur un domaine HTTPS ;
- remplacer SQLite par MySQL ou PostgreSQL ;
- utiliser Laravel Sanctum ou Passport ;
- ajouter notifications push ;
- ajouter paiement en ligne ;
- créer un dashboard web complet ;
- ajouter CI/CD, monitoring et sauvegardes.

A Comptes de démonstration

TABLE A.1 – Comptes seedés

Email	Rôle	Mot de passe
yasmine@monpf.fr	client	password
admin@monpf.fr	admin	password
serveur@monpf.fr	serveur	password
livreur@monpf.fr	livreur	password

B Fichiers importants

TABLE B.1 – Fichiers à connaître

Fichier	Rôle
<code>mon_pfapp/lib/app/mon_pf_app.dart</code>	État global et navigation.
<code>mon_pfapp/lib/data/api_client.dart</code>	Client HTTP.
<code>mon_pfapi/routes/api.php</code>	Routes REST.
<code>mon_pfapi/config/cors.php</code>	Configuration Web/CORS locale.
<code>docs/yassmine-guide/index.html</code>	Guide interactif et assistant debug.
<code>docs/demo-prod-runbook.md</code>	Guide rapide demo-prod.
<code>docs/backend-api.md</code>	Documentation backend.